

Experiment 1: Object Detection and Recognition

Pencil and Tennis-Ball Detection System Based on YOLO26n, Jetson, and ROS 2

Course:	Robotic Systems Integration Laboratory
Experiment:	Object Detection and Recognition
Name:	Haoran Xiao
Student ID:	24020036066
Period:	August 24–30, 2026
GitHub:	xhr-CHN/jetson-ros2-object-detection

August 30, 2026

Abstract

This experiment addresses desktop-object detection using pencils and tennis balls. It covers dataset organization, unified YOLO-format annotation, YOLO26n training, independent test-set evaluation, Jetson Orin deployment, and ROS 2 publication. Four public sources from Mendeley Data, Kaggle, and Roboflow Universe were merged. Five pencil subclasses were mapped to `pencil`, and all tennis-ball labels to `tennis_ball`. The final dataset contains 895 images and 1,048 objects: 605 training, 171 validation, and 119 test images. The test set was not used for training or best-weight selection.

The model completed a formal 100-epoch augmented run on an NVIDIA GeForce RTX 4060 Laptop GPU; the best weights occurred at epoch 87. Validation Precision, Recall, $\text{mAP}@0.5$, and $\text{mAP}@0.5:0.95$ were 0.96338, 0.97329, 0.98185, and 0.88025. Independent-test results were 0.98877, 0.90186, 0.97148, and 0.87761. PT, ONNX, and TensorRT Engine formats were produced. On the Jetson, the camera image topic averaged 13.795 FPS and detection FPS averaged 16.657 FPS.

Only the second on-device test was used for final acceptance. With seed 20260830, it selected 20 pencil and 20 tennis-ball images from the held-out test set. Thirty-nine of 40 were correct: 97.5% overall, 95.0% for pencils, and 100.0% for tennis balls. The only error was a missed red pencil; all 40 results were retained. The code, data, models, video, instructions, and incremental Git history are public. All course requirements were met.

Keywords: object detection; YOLO26n; Jetson Orin; TensorRT; ROS 2; pencil; tennis ball

Contents

Abstract	I
Chapter 1 Experiment Requirements and Overall Completion	1
1.1 Experiment Requirements	1
1.2 Acceptance Criteria	1
Chapter 2 System Design and Key Technologies	1
2.1 Overall Workflow	1
2.2 YOLO26n Model Selection	2
2.3 Evaluation Metrics	2
Chapter 3 Dataset Construction and Annotation Processing	2
3.1 Data Sources and Licensing	2
3.2 Cleaning, Class Mapping, and Dataset Splits	3
Chapter 4 Model Training and Validation	4
4.1 Training Environment and Parameters	4
4.2 Training Process	5
4.3 Best Validation Metrics and Model Finalization	5
Chapter 5 Independent Test-Set Evaluation	6
5.1 Overall and Per-Class Results	6
5.2 PR Curve and Confusion Matrix	6
5.3 Detection Examples	7
Chapter 6 Model Export and Jetson Deployment	8
6.1 Three Model Formats	8
6.2 On-Device Environment and Real-Time Results	8
Chapter 7 ROS 2 Node and Topic Design	9
7.1 Detection Node	9
7.2 Message Content	10
Chapter 8 Final 40-Item Acceptance Test and Error Analysis	10
8.1 Test Method	10
8.2 Results	11
8.3 Sole Error Case	11
Chapter 9 GitHub Process Management and Submission Completeness	13
9.1 Public Repository and Incremental Commits	13
9.2 Final Submission Materials	15
Chapter 10 Conclusions and Future Improvements	15
References	16

Chapter 1 Experiment Requirements and Overall Completion

1.1 Experiment Requirements

The course requires at least two desktop-object classes, data collection or public-data preparation and annotation, model training, real-time Jetson display of classes, boxes, and confidence, and ROS 2 publication. Acceptance requires at least 20 tested objects, accuracy of at least 80%, Jetson speed of at least 5 FPS, and preserved results and representative errors. Required submissions are the dataset, model, programs, result video, instructions, and LaTeX report.

This project uses `pencil` and `tennis_ball`. Pencils are slender and tennis balls approximately round, providing complementary scales and aspect ratios. Both are easy to obtain for USB-camera and Jetson demonstrations.

1.2 Acceptance Criteria

Table 1 Course requirements and actual results

Requirement	Verification	Actual result	Status
At least two classes	Configuration and video	Pencil and tennis ball	Pass
At least 20 objects	Second fixed Jetson test	40 items, 20 per class	Pass
Accuracy at least 80%	Count 40 predictions	39/40, 97.5%	Pass
Speed at least 5 FPS	Topic and node samples	13.795/16.657 FPS	Pass
Boxes, classes, confidence	Final video and code	Both classes displayed	Pass
ROS 2 publication	Node and topic records	Three topics active	Pass
Preserved tests/errors	Directory integrity	40 images, one error	Pass

The 97.5% value is classification-style accuracy on 40 fixed items, not object-detection mAP. Offline TensorRT speed is also distinct from end-to-end camera, inference, rendering, and ROS 2 throughput. The report keeps these definitions separate.

Chapter 2 System Design and Key Technologies

2.1 Overall Workflow

The workflow is data–training–evaluation–export–deployment–publication–acceptance. Windows handles data preparation, 100-epoch training, and independent testing. The Jetson

generates its environment-specific TensorRT Engine, performs USB-camera detection, runs ROS 2, and records acceptance evidence.

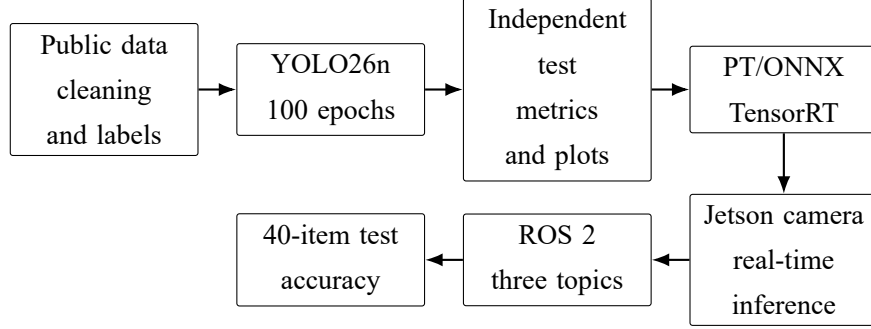


Figure 1 Overall technical workflow

2.2 YOLO26n Model Selection

YOLO unifies classification and box regression in a real-time single-stage detector. YOLO26 uses an end-to-end design and a one-to-one head. YOLO26n is the smallest scale and has low computation at 640-pixel input, which suits an edge device[1, 2]. The nano model was selected to balance accuracy and stable Jetson speed. For image size W, H and box corners $(x_1, y_1), (x_2, y_2)$, normalized YOLO coordinates are

$$x_c = \frac{x_1 + x_2}{2W}, \quad y_c = \frac{y_1 + y_2}{2H}, \quad w = \frac{x_2 - x_1}{W}, \quad h = \frac{y_2 - y_1}{H}. \quad (1)$$

Each label line contains a class ID and (x_c, y_c, w, h) in $[0, 1]$.

2.3 Evaluation Metrics

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F1 = \frac{2PR}{P + R}. \quad (2)$$

AP is the area under the Precision–Recall curve. mAP@0.5 uses IoU 0.5; mAP@0.5:0.95 averages thresholds from 0.5 to 0.95 in steps of 0.05. Course acceptance accuracy is computed separately from correct item-level outcomes.

Chapter 3 Dataset Construction and Annotation Processing

3.1 Data Sources and Licensing

Four public sources were combined. Mendeley Tennis Ball v2 supplied 150 images under CC BY 4.0, after removal of one exact duplicate. Kaggle Pencil supplied 300 images; JSON/COCO boxes were converted to YOLO, but the downloaded page did not specify a license, which remains a documented compliance limitation. Roboflow Pencil v2 supplied

94 images under CC BY 4.0, with five pencil subclasses merged. Roboflow Tennis ball v1 supplied 352 images under Public Domain. Full URLs, licenses, and processing records are in `dataset/SOURCES.md` and `dataset/dataset_summary.json`[5, 6, 7, 8].

Table 2 Public data sources and processing

Source	Images	License	Processing
Mendeley Tennis Ball v2	150	CC BY 4.0	Remove one duplicate
Kaggle Pencil	300	Unspecified	Convert JSON/COCO to YOLO
Roboflow Pencil v2	94	CC BY 4.0	Merge five subclasses
Roboflow Tennis ball v1	352	Public Domain	Map to tennis_ball

3.2 Cleaning, Class Mapping, and Dataset Splits

All images were checked for readability and all labels converted to YOLO. Every pencil appearance and color maps to class 0; every tennis ball maps to class 1. Source prefixes prevent filename collisions.

Listing 1 Core final dataset configuration

```

1 path: .
2 train: images/train
3 val: images/val
4 test: images/test
5 names:
6   0: pencil
7   1: tennis_ball

```

The 119 test images were excluded from training and best-epoch selection.

Table 3 Final YOLO dataset statistics

Split	Images	Pencil objects	Tennis-ball objects
train	605	317	387
val	171	100	94
test	119	80	70
Total	895	497	551

The class counts are reasonably balanced. The test set contains 60 pencil and 59 tennis-ball images. Different source backgrounds may still reduce cross-domain generalization, motivating more genuine mixed desktop scenes.

Chapter 4 Model Training and Validation

4.1 Training Environment and Parameters

Formal Windows training uses `src/train_yolo26n_windows.py`. It verifies CUDA and never silently falls back to CPU. Training used an RTX 4060 Laptop GPU, 640 input, batch 8, and 100 epochs. Arguments and per-epoch metrics are in `results/training/yolo26n_augmented/args.yaml` and `results/training/yolo26n_augmented/results.csv`.

Table 4 Formal training configuration

Item	Configuration
Model	YOLO26n pretrained weights
Task	Two-class detection
Input	640×640
Epochs	100
Batch size	8
Device	NVIDIA GeForce RTX 4060 Laptop GPU
Data	<code>dataset/data-windows.yaml</code>
Output	<code>runs/pencil_tennis_yolo26n_augmented</code>

4.2 Training Process

Figure 2 is the original color result from all 100 epochs. Losses declined and validation metrics stabilized. The best weights occurred at epoch 87 and are not from an interrupted run.

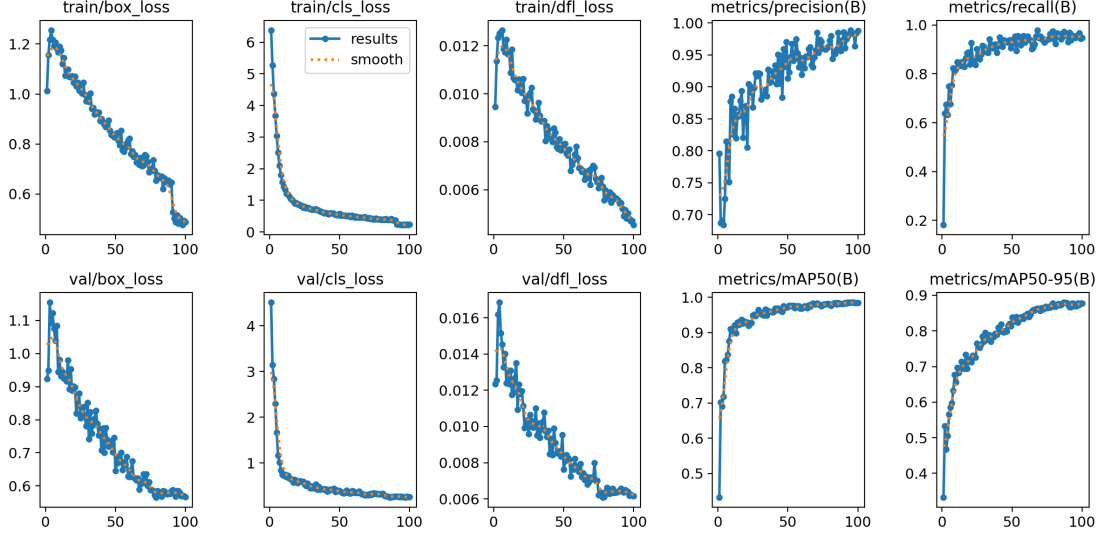


Figure 2 Complete 100-epoch training and validation curves

4.3 Best Validation Metrics and Model Finalization

Table 5 Best-weight validation metrics

Metric	Value
Precision	0.96338
Recall	0.97329
mAP@0.5	0.98185
mAP@0.5:0.95	0.88025
Best epoch	87

The weights were copied to `models/pencil_tennis_yolo26n_best.pt`. The model card records parameters, results, environment, limitations, and SHA-256. Validation does not replace independent testing or course acceptance.

Chapter 5 Independent Test-Set Evaluation

5.1 Overall and Per-Class Results

After model finalization, `src/evaluate_yolo26n_windows.py` evaluated 119 held-out images. Overall Precision was 0.98877, Recall 0.90186, mAP50 0.97148, and mAP50–95 0.87761. Pencil localization was more stable; lower tennis-ball Recall indicates missed small objects or difficult backgrounds.

Table 6 Independent test-set metrics

Class	Images	Objects	P	R	mAP50	mAP50–95
pencil	60	80	1.0000	.9466	.9941	.9542
tennis_ball	59	70	.9775	.8571	.9489	.8010
Overall	119	150	.9888	.9019	.9715	.8776

5.2 PR Curve and Confusion Matrix

The original-color Ultralytics PR curve in Figure 3 shows a favorable balance. The normalized matrix in Figure 4 gives approximately 0.99 correct pencils and 0.89 correct tennis balls. Errors are mainly missed detections assigned to background, not cross-class confusion.

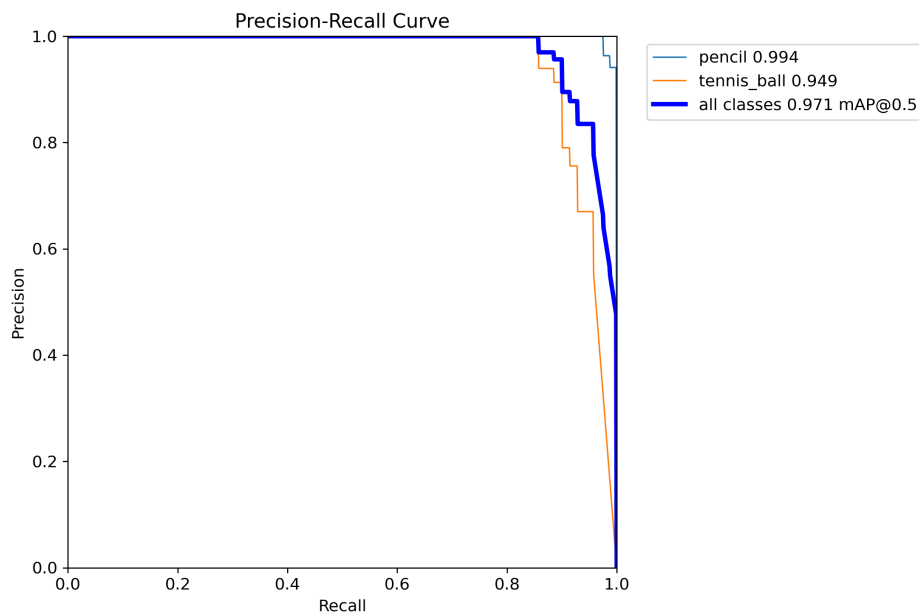


Figure 3 Independent test-set PR curve

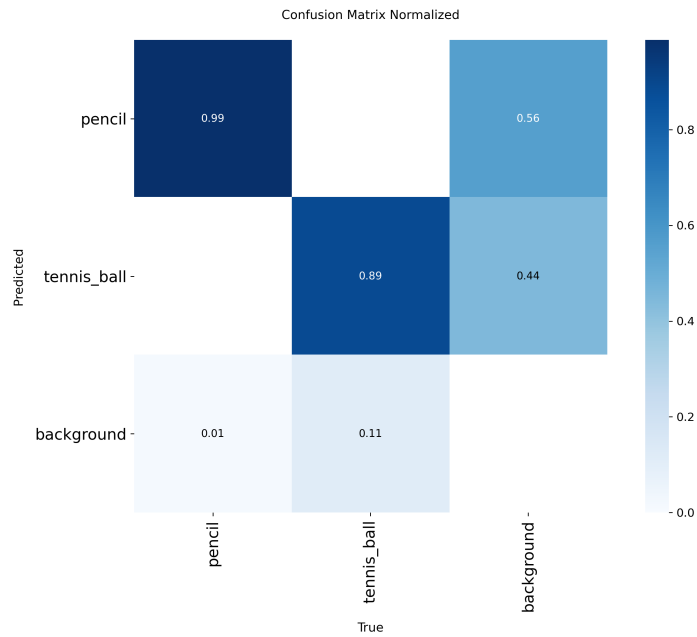


Figure 4 Independent test-set normalized confusion matrix

5.3 Detection Examples

These are genuine predictions from the 119-image test output and show boxes, labels, and confidence.



Figure 5 Test-set pencil detection



Figure 6 Test-set tennis-ball detection

Chapter 6 Model Export and Jetson Deployment

6.1 Three Model Formats

PT supports reproduction and further training, ONNX supports framework exchange, and TensorRT Engine supports efficient inference[4]. An Engine depends on JetPack, TensorRT, CUDA, GPU architecture, and build settings and must be regenerated when the environment changes.

Table 7 Final model files and integrity records

File	Size/bytes	Abbreviated SHA-256
pencil_tennis_yolo26n_best.pt	5,415,023	DA747F4ED471BE2A...A4063D
pencil_tennis_yolo26n_best.onnx	9,761,287	4616C124B10E4B2C...C2401
pencil_tennis_yolo26n_best.engine	7,863,922	BBFFFC6D678DDF86...DC0E3

6.2 On-Device Environment and Real-Time Results

The aarch64 Jetson Orin ran Jetson Linux R36.4.3, CUDA 12.6, TensorRT 10.3.0, PyTorch 2.10.0, and ROS 2 Humble[3]. USB capture used V4L2 MJPG at 640×480 with buffer size 1. Figure 7, from second 8 of the final 29.76-second H.264/AAC video, shows both classes, boxes, confidence, and 16.7 FPS. The video is `results/jetson/2026-08-30/jetson_demo.mp4`.

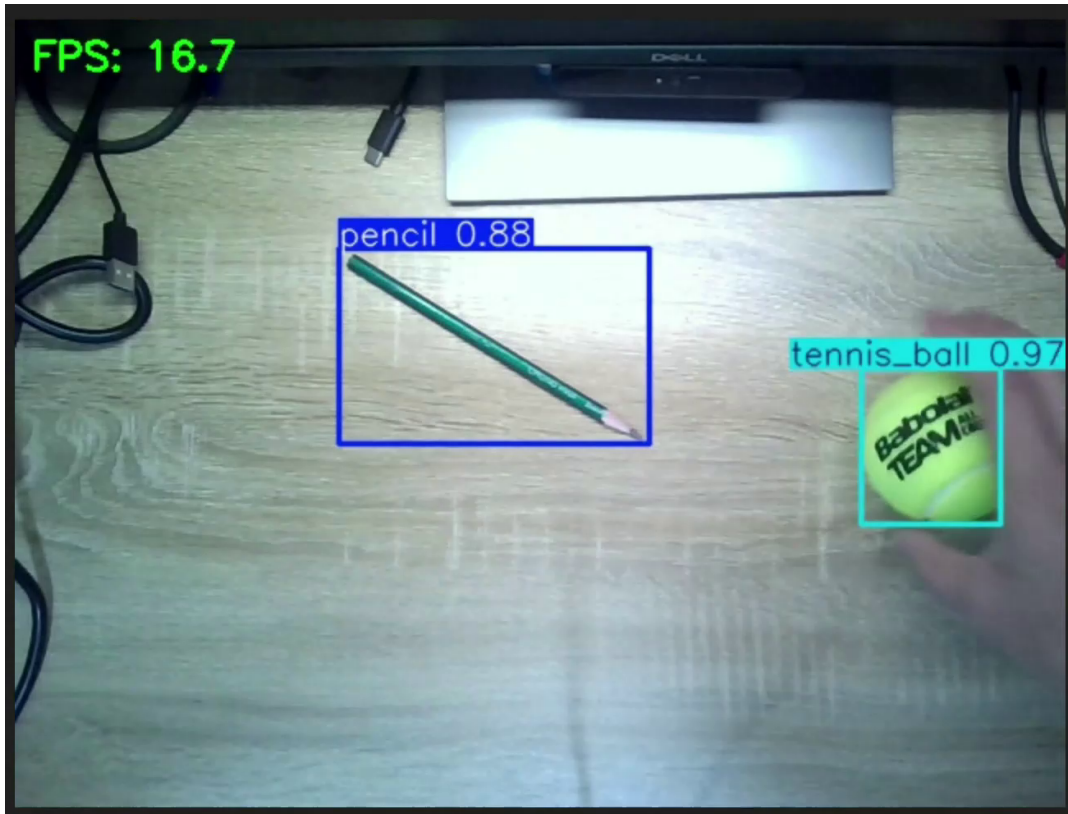


Figure 7 Jetson real-time detection at second 8

Table 8 Jetson real-time speeds

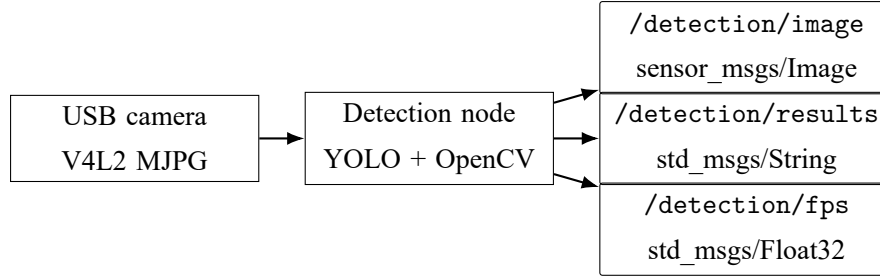
Measurement	Samples	Minimum	Average	Maximum
/detection/image rate	5	13.369	13.795	14.092
/detection/fps data	46	15.421	16.657	17.971

Image-topic rate represents end-to-end output more closely; the FPS topic uses per-frame processing time. Both minimums exceed 5 FPS.

Chapter 7 ROS 2 Node and Topic Design

7.1 Detection Node

The package is under `ros2_ws`. `detector_node.py` handles capture, inference, parsing, drawing, and three publications. Node `/pencil_tennis_detector` accepts model, camera, confidence, input-size, CUDA-device, and timer parameters.

**Figure 8** ROS 2 node and topics

7.2 Message Content

Table 9 ROS 2 topic design

Topic	Type	Content
/detection/image	sensor_msgs/Image	Annotated image with FPS
/detection/results	std_msgs/String	Per-frame JSON detections
/detection/fps	std_msgs/Float32	Processing speed

The JSON contains timestamp, frame number, image size, classes, confidence, and pixel boxes. One genuine tennis-ball record has confidence 0.958984375.

Listing 2 Detection-result JSON example

```

1 {"frame_id":6710,"image_width":640,"image_height":480,
2  "detections":[{"class_id":1,"class_name":"tennis_ball",
3  "confidence":0.958984375,
4  "bbox":{"x1":145.625,"y1":168.0,"x2":313.5,"y2":332.0}}]}

```

Chapter 8 Final 40-Item Acceptance Test and Error Analysis

8.1 Test Method

Two offline attempts were preserved. The first, 10 images per class, was preliminary. Only attempt 2 is final. Seed 20260830 selected 40 unique images only from dataset/images/test: 20 pencils followed by 20 tennis balls, each with one ground-truth object. The formal TensorRT Engine used confidence 0.25. The highest-confidence detection determined the class; no detection was an error. All evidence is in results/jetson/2026-08-30/acceptance/attempt_02.

8.2 Results

Table 10 Second and final Jetson acceptance test

Class	Tested	Correct	Errors	Accuracy
pencil	20	19	1	95.0%
tennis_ball	20	20	0	100.0%
Total	40	39	1	97.5%

$$\text{Accuracy} = \frac{39}{40} \times 100\% = 97.5\%. \quad (3)$$

This is 17.5 percentage points above the threshold. Offline TensorRT averaged 31.7906 FPS and reached a minimum of 30.1659 FPS; it excludes camera capture and ROS 2 publication.

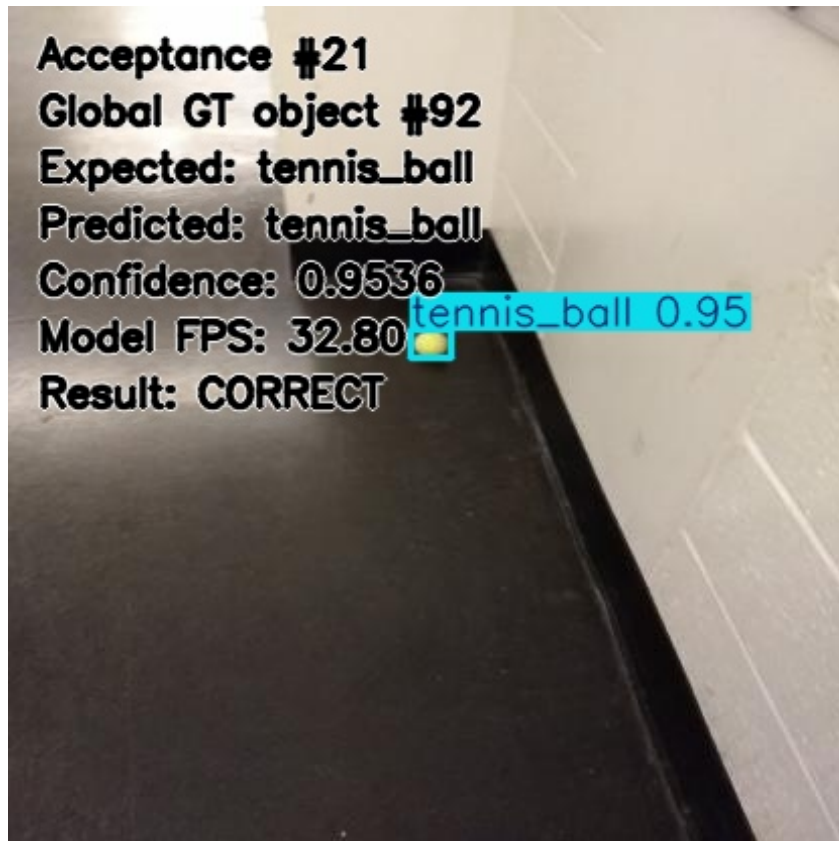


Figure 9 Correct tennis-ball acceptance case

8.3 Sole Error Case

The only error was a missed pencil_red4.jpg. No detection exceeded 0.25, so the prediction was empty with confidence zero. It was not removed, replaced, or resampled.

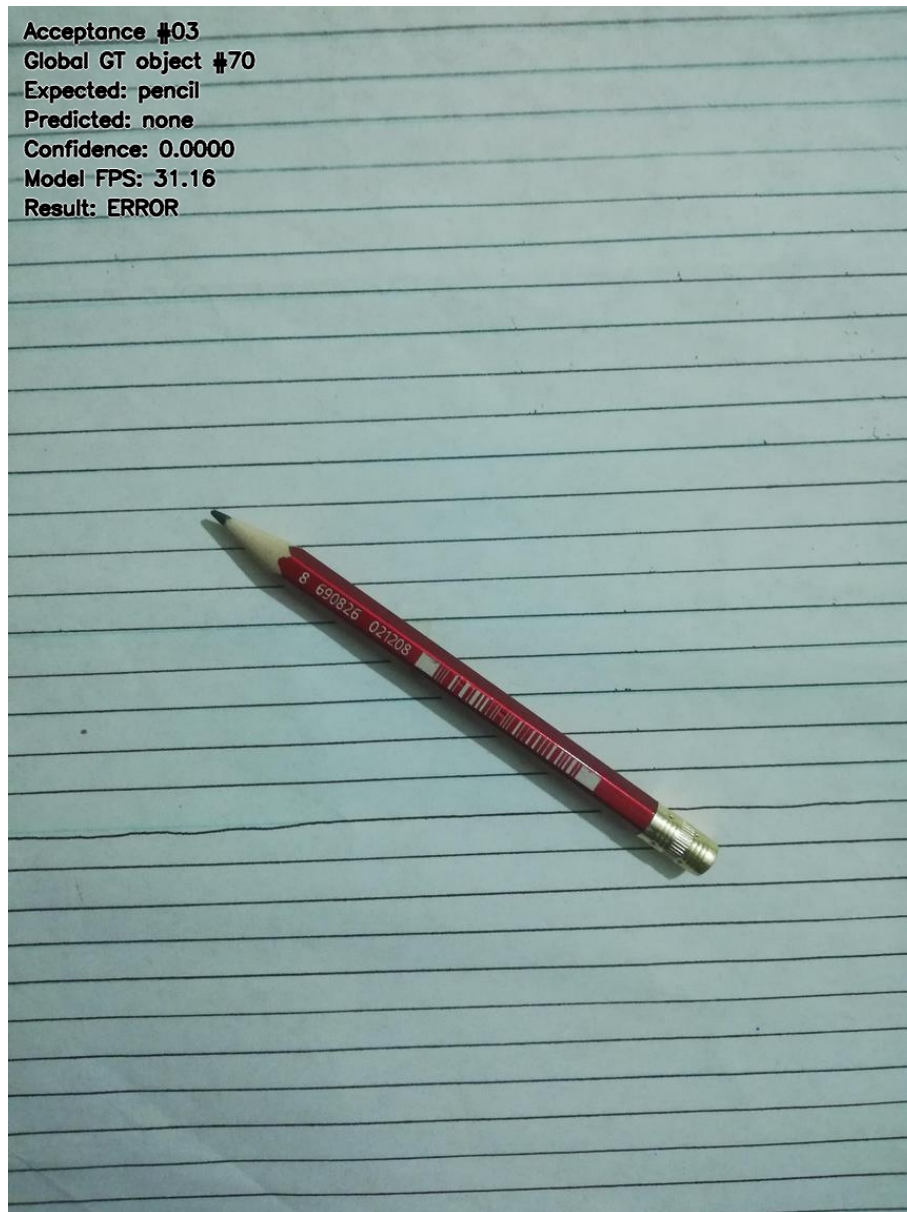


Figure 10 Sole error: missed red pencil

The pencil is thin and small, with interference from horizontal background lines. The main risk is assignment to background, not class confusion. Future improvements include more low-contrast slender objects, mixed scenes, threshold calibration, and higher resolution. The test error was not used to retrain and retest, avoiding leakage.

Chapter 9 GitHub Process Management and Submission Completeness

9.1 Public Repository and Incremental Commits

The public repository is `https://github.com/xhr-CHN/jetson-ros2-object-detection`. It organizes code, data, models, plots, Jetson evidence, video, instructions, and this report. Figure 11, captured August 30, 2026, shows the main branch and 30 commits at that time.

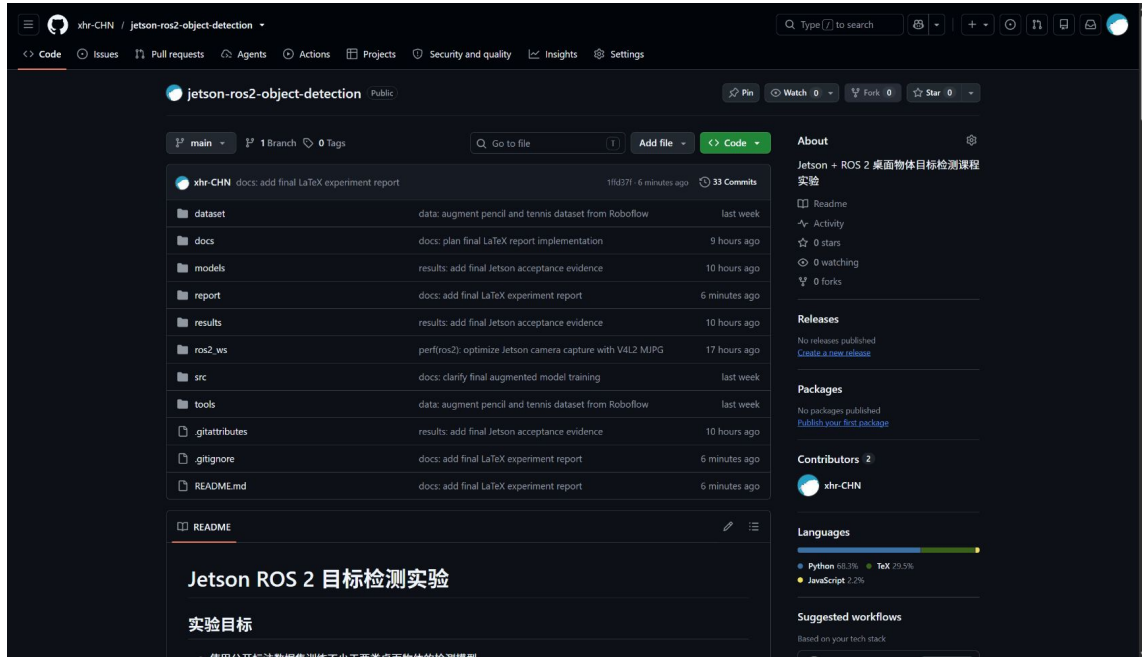


Figure 11 Public GitHub repository

Work was committed incrementally by data, training, inference, ROS 2, export, Jetson evidence, video, and acceptance result, rather than in one bulk commit. Later report commits are not shown in the historical screenshot.

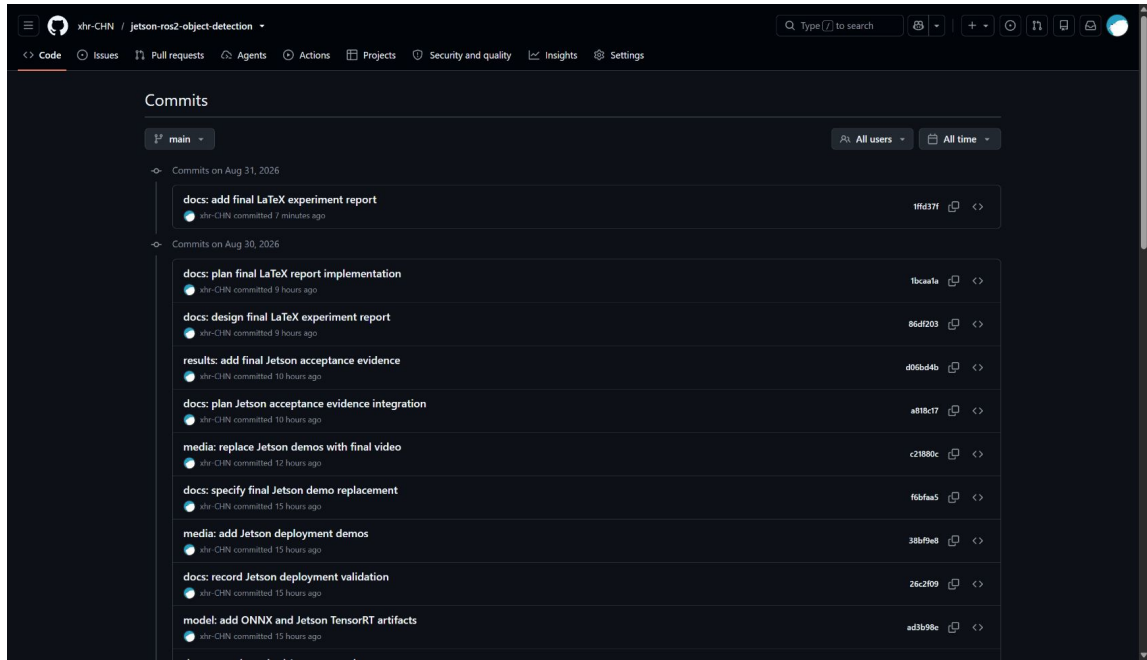


Figure 12 Continuous main-branch commit history

Table 11 Representative Git commits

Commit	Type	Content
4ee5a04	data	Merge Roboflow supplemental data
2e385b3	train	Add formal augmented training
9e616f5	feat	Add augmented-model webcam detection
77a9956	model	Finalize best weights from 100 epochs
85923cb	test	Run independent test
2e907f8	docs	Clarify complete 100-epoch training
2e15f0d	perf(ros2)	Optimize V4L2 MJPG capture
ad3b98e	model	Add ONNX and TensorRT artifacts
26c2f09	docs	Record system, topics, and speed
c21880c	media	Add final Jetson video
a818c17	docs	Plan acceptance evidence
d06bd4b	results	Commit final attempt-2 evidence

9.2 Final Submission Materials

Table 12 Final GitHub materials

Path	Content
dataset/	895 images, labels, configuration, sources
models/	PT, ONNX, Engine, model card
src/	Windows training, evaluation, webcam code
ros2_ws/	Detector, recorder, launch, tests
results/training/	100-epoch arguments, CSV, plots
results/test/	119 predictions, metrics, PR, matrix
results/jetson/	Environment, speed, acceptance, error, video
README.md	Operating instructions
report/	LaTeX, references, figures, PDF

The unspecified Kaggle pencil license remains a compliance limitation. The repository preserves its source and risk statement for review and possible replacement.

Chapter 10 Conclusions and Future Improvements

The project completed the full path from data to edge deployment: 895 images and 1,048 objects; 100 training epochs with best epoch 87; independent-test mAP@0.5 of 0.97148 and mAP@0.5:0.95 of 0.87761; Jetson camera-topic average of 13.795 FPS; three ROS 2 outputs; and 39/40 final acceptance accuracy of 97.5%. CSV, JSON, plots, matrices, images, logs, video, and Git history support every result.

Validation, independent testing, and acceptance serve different purposes. TensorRT Engines are environment-dependent, so hashes, software, and hardware must be recorded together. Preserving the sole missed detection and continuous commit history makes the work traceable. Future work should add low-light, occluded, complex, and mixed-class desktop scenes; complement the fixed test with physical camera acceptance; and replace JSON strings with a custom ROS 2 message plus time synchronization and coordinate transforms. A JetPack or Jetson change requires Engine regeneration and repeated speed and accuracy tests.

References

- [1] Jocher G, Qiu J, Liu M, et al. Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models[EB/OL]. arXiv:2606.03748, 2026. <https://arxiv.org/abs/2606.03748>.
- [2] Ultralytics. Ultralytics YOLO26 Documentation[EB/OL]. 2026[2026-08-30]. <https://docs.ultralytics.com/models/yolo26/>.
- [3] Open Robotics. ROS 2 Humble Documentation[EB/OL]. [2026-08-30]. <https://docs.ros.org/en/humble/>.
- [4] NVIDIA Corporation. NVIDIA TensorRT Documentation[EB/OL]. [2026-08-30]. <https://docs.nvidia.com/deeplearning/tensorrt/latest/>.
- [5] Jagtiani K. Object Detection–Tennis Ball, Version 2[DS/OL]. Mendeley Data, 2021. DOI:10.17632/ppr8rdw98w.2.
- [6] Saglam F. Pencil Dataset[DS/OL]. Kaggle, 2023[2026-08-24]. <https://www.kaggle.com/datasets/greysky/pencil-dataset>.
- [7] Roboflow Universe. Pencil Dataset, Version 2[DS/OL]. [2026-08-24]. <https://universe.roboflow.com/workspace1-1gbmx/pencil-tvhes/dataset/2>.
- [8] Roboflow Universe. Tennis Ball Dataset, Version 1[DS/OL]. [2026-08-24]. <https://universe.roboflow.com/tennis-3110a/tennis-ball-icifx/dataset/1>.